TAHX_1

Processor Reference Manual

Architecture, modèle programmeur et jeu d'instructions

ISA V8

Édition 1.0 - octobre 2026

Tlaloc Ada Hardware eXecutor - version 1

Manuel de référence architectural dérivé de la spécification normative V8 du 29 septembre 2026.

Vincent Morin - Université de Bretagne Occidentale - 2026

Statut du document

Ce manuel est une **présentation de référence destinée au lecteur humain** de l'architecture TAHX_1 et de son jeu d'instructions HX. Il réorganise et explique la spécification V8 dans la forme d'un manuel de processeur classique.

Autorité normative

En cas d'ambiguïté ou de contradiction, la spécification *LLIR_hardware_support_V8.txt* reste normative. La V8 désigne `tx_run` comme référence exécutable, sous réserve de sa liste de vérification. Ce manuel ne crée ni opcode, ni sémantique, ni contrainte nouvelle.

La révision V8 ne modifie pas l'encodage de la V7 et fixe notamment la priorité des fautes, les réserves de piles, les règles de `FEXP`, les conversions de virgule fixe et le statut des opcodes réservés. Elle ne conserve aucune question ouverte dans la définition de l'ISA proposée pour le gel.

Conventions de lecture

— Les effets de pile utilisent la notation `( avant - après )`, avec le sommet à droite.
— DSP, RSP, CSP, HP, PC et DISPLAY désignent l'état architectural.
— `M64[a]` désigne un mot de 64 bits en mémoire à l'adresse `a`.
— Les mots « doit », « interdit », « faute » et « réservé » reprennent un comportement architectural, et non un choix d'implantation.

Vue d'ensemble en une page

TAHX_1 est une machine à pile de 64 bits conçue pour exécuter directement les images HX issues de la LLIR de TLALOC. Le code est un flux unique d'instructions de longueur variable (1 à 9 octets). Le premier octet suffit toujours à connaître la longueur de l'instruction. Les données sont adressées par octet, en petit-boutiste, avec accès non alignés autorisés.

Le modèle programmeur possède une pile data, une pile de retours séparée, une co-pile, un tas descendant et un DISPLAY de quinze niveaux statiques. Les opérations entières de base sont contrôlées : elles ne débordent jamais silencieusement. Les flottants sont des IEEE 754 binary64. Les fautes sont précises et sont reliées aux exceptions Ada 83 par une table de vecteurs et un trampoline du run-time.

Caractéristiques architecturales essentielles

Mot machine	64 bits
Cellule de pile data	64 bits
Espace d'adressage architectural	adresses sur 64 bits ; adresses valides de plateforme $< 2^{63}$
Boutisme des données	petit-boutiste
Alignement	non imposé
Code	un seul flux, longueur variable, maximum 9 octets
Niveaux statiques	DISPLAY[0..14]
Flottants	IEEE 754 binary64
Déroutements	services TRAP, interruptions 32..63, fautes 128..159
Image exécutable	format HX, signature `TLALOCHX`

Table des matières

Statut du document i

Vue d'ensemble en une page ii

1 Architecture générale 1

1.1 Position de TAHX_1 dans la chaîne TLALOC . . . 1

1.2 Vue fonctionnelle . . . 1

1.3 Principes structurants . . . 1

2 Modèle programmeur 3

2.1 État architectural . . . 3

2.2 Organisation des piles . . . 3

2.3 Frames et niveaux statiques . . . 4

2.4 Mémoire . . . 4

2.5 Programme valide . . . 4

3 Codage des instructions 5

3.1 Flux unique et longueur déterminée par l'opcode . . . 5

3.2 Les quatre familles . . . 5

3.3 Formats d'opérandes . . . 6

3.4 Ordre des octets . . . 6

3.5 Immédiats et branches . . . 6

4 Adressage, frames et accès mémoire 7

4.1 Calcul de l'adresse effective . . . 7

4.2 Notation @ et @@ . . . 7

4.3 Chargements et rangements . . . 7

4.4 CHK . . . 7

4.5 Replis d'assemblage . . . 8

5 Instructions entières, logiques et de bits 9

5.1 Règle de débordement . . . 9

5.1.1 Logique et décalages . . . 9

5.1.2 Comparaisons entières . . . 9

5.1.3 Arithmétique entière . . . 10

5.1.4 Champs de bits . . . 10

5.1.5 Virgule fixe 10

6 Instructions flottantes **11**

6.1 Modèle IEEE 754 11

6.1.1 Arithmétique et conversions 11

6.1.2 Comparaisons flottantes et signe 11

7 Pile, blocs et allocation **12**

7.0.1 Manipulation de pile 12

7.0.2 Blocs mémoire 12

7.0.3 Allocation 12

7.1 Cellules de calcul et mémoire 12

8 Frames, appels et contrôle de flot **14**

8.1 LINK et UNLINK 14

8.2 Appels et retours 14

8.3 EXC_MACH et EXC_RAISE 14

9 Fautes, déroutements et interruptions **15**

9.1 Trois sources de déroutement 15

9.2 Faute précise 15

9.3 Table des fautes 16

9.4 Priorité des fautes 16

9.5 Réserves de piles 17

9.6 Interruptions 17

10 TRAP, services et contextes de tâche **18**

10.1 TRAP 18

10.2 Services numérotés 18

10.3 CTX_SAVE et CTX_RESTORE 19

10.4 SET_IMASK 19

10.5 Lien avec les entrées de tâche Ada 83 19

11 Image HX et démarrage **20**

11.1 Format HX 20

11.2 Disposition conceptuelle 20

11.3 État initial 21

11.4 Bloc de démarrage de la plateforme 21

12 Référence rapide des familles mémoire **22**

12.1 Famille B : accès direct 22

12.2 Famille C : accès indirect 22

12.3 Comparaison lexicographique 22

A Carte complète de l'espace d'opcodes **23**

B Table numérotée des instructions V8 **26**

C Formats et longueurs - aide-mémoire **31**

C.1 Formats 31

C.2 Règles d'adresse 31

C.3 Drapeaux et registres de déroutement 31

D Glossaire **32**

Référence normative **33**

Index fonctionnel **34**

Chapitre 1

Architecture générale

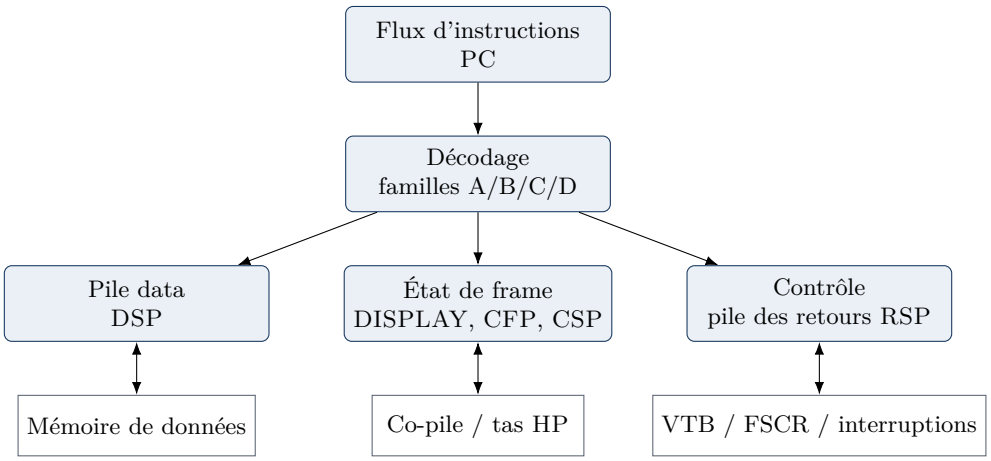
1.1 Position de TAHX_1 dans la chaîne TLALOC

TAHX_1 est l'exécuteur matériel du niveau LLIR/HX : le compilateur et son expander produisent des opérations dont la sémantique est matérialisée par le jeu d'instructions décrit ici. Le processeur ne cherche donc pas à reproduire une architecture de registres généraliste ; il expose directement un modèle à pile, des niveaux statiques, des opérations de frame et les mécanismes nécessaires aux exceptions et au tasking Ada 83.

Architecture et micro-architecture

Le manuel décrit ce qu'un programme HX peut observer. Une implantation peut mettre des cellules de pile en registres, retarder certaines écritures ou organiser la pile des retours hors du chemin des données, à condition que le comportement soit exactement celui du modèle architectural. Le pipeline, le nombre d'unités, les caches et l'ordonnancement ne font donc pas partie de l'ISA.

1.2 Vue fonctionnelle



1.3 Principes structurants

— **Pile d'évaluation 64 bits.** Les opérandes et résultats des expressions transitent par la pile data.
— **Niveaux statiques.** DISPLAY[0..14] donne un accès direct aux frames lexicales, ce qui correspond au modèle de portée d'Ada.
— **Retour séparé.** Les adresses de retour ne partagent pas la pile data : CALL/CALLI/TRAP/interruptions utilisent la pile des retours.
— **Code immuable après démarrage.** Une implantation peut donc conserver des formes décodées sans protocole d'invalidation.

— **Fautes précises.** Une instruction fautive ne laisse aucun effet propre ; le handler observe l'état antérieur à l'instruction.
— **Contrôle arithmétique.** Les opérations entières signées de base lèvent une faute de débordement au lieu de boucler modulo 2^{64}.

Chapitre 2

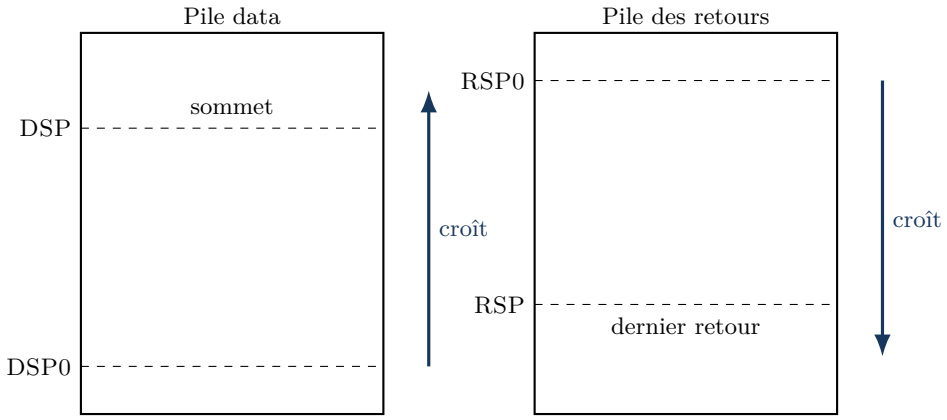
Modèle programmeur

2.1 État architectural

Élément	Rôle
PC	compteur ordinal, dans l'unique flux de code
DSP	adresse de la cellule au sommet de la pile data
RSP	adresse de la dernière adresse empilée sur la pile des retours
CFP	frame courant de co-pile
CSP	premier octet libre de la co-pile
HP	frontière basse du tas déjà alloué
DISPLAY[0..14]	frame pointers des quinze niveaux statiques
DR	bit « déroutement en cours »
FPC, FCODE	adresse et code de la dernière faute ou interruption
VTB, FSCR	table des vecteurs et zone d'exposition de FPC/FCODE
LIM_DSP, LIM_RSP, LIM_CSP, LIM_HP	limites des piles et du tas
IMASK	masque 32 bits des interruptions externes

Le code `lvl=1111` ne désigne pas un seizième niveau : il signifie « adresse prise sur la pile data ». Il reste donc quinze niveaux effectifs, 0 à 14.

2.2 Organisation des piles



La pile data croît vers les adresses hautes. Un *push* fait `DSP := DSP + 8` puis écrit la cellule ; un *pop* lit `M64[DSP]` puis retranche 8. La pile des retours croît en sens inverse : `push_retour` retranche 8 avant l'écriture.

La co-pile croît vers le haut et CSP désigne le premier octet libre. Le tas croît vers le bas et HP désigne la frontière basse de la zone déjà allouée.

2.3 Frames et niveaux statiques

Les paramètres sont situés à des déplacements négatifs à partir de -8 ; les variables locales à des déplacements positifs à partir de +8. DISPLAY[lvl] pointe sur la cellule où le frame pointer antérieur a été sauvegardé.

`LINK`, `UNLINK` et `UNLINKR` gèrent à la fois la pile data et le chaînage de co-pile. La distinction entre `UNLINK` et `UNLINKR` est importante : le premier ne rend pas la co-pile du frame, afin de permettre l'évasion d'un résultat tableau ; le second la rend.

2.4 Mémoire

La mémoire est adressée par octets. Les accès 16, 32 et 64 bits sont petit-boutistes. Un accès non aligné est architecturalement valide et doit donner le même résultat qu'une suite d'accès d'un octet ; il peut seulement être plus lent.

Un accès n'est valide que si tous ses octets le sont. Une adresse valide de plateforme est inférieure à 2^{63}. Les instructions de calcul d'adresse de la machine peuvent former leurs sommes modulo 2^{64} ; c'est l'accès mémoire final qui décide de la validité.

Code non modifiable

Après le début de l'exécution, modifier un octet appartenant à une instruction donne un comportement non spécifié. Les données placées en ligne dans le flux, mais contournées par une branche, ne sont pas des instructions et ne sont pas concernées par cette règle.

2.5 Programme valide

Toute adresse atteignable dynamiquement par le contrôle doit être le début d'une instruction : cible de branche ou de CALL, retour, DISPATCH d'exception, vecteur, point d'entrée ou PC restauré depuis un contexte. La machine ne peut pas distinguer une cible valide d'un milieu d'instruction ; l'image HX peut toutefois fournir une table qui permet à l'émulateur de diagnostiquer ce cas.

Chapitre 3

Codage des instructions

3.1 Flux unique et longueur déterminée par l'opcode

Le code HX est un flux unique. La longueur d'une instruction est une fonction de son seul premier octet. Aucun champ complémentaire n'intervient dans ce calcul. Cette propriété permet à un décodeur de repérer plusieurs frontières d'instruction dans un bloc d'octets sans interpréter d'abord les opérandes.

Longueurs

Famille A	sans complément	1 octet
B16 / B24	accès direct	3 / 4 octets
C24 / C32	accès indirect	4 / 5 octets
LI	immédiat 8/16/32/64	2 / 3 / 5 / 9 octets
BR8/16/24/32	branche	2 / 3 / 4 / 5 octets
TRAP, UNLINK, UNLINKR	D8	2 octets
CALL, RTD n, EXC_RAISE	D24	4 octets

Longueur maximale : **9 octets**, pour `LI imm64`. Un opcode réservé n'a pas de longueur architecturale : son décodage s'arrête sur le premier octet et lève la faute 137.

3.2 Les quatre familles

OP[7..6]	Famille	Usage
00	A	opérations sans complément : ALU, pile, flottants, blocs
01	B	accès direct : `lvl + disp`
10	C	accès indirect : `lvl + disp + ofs`
11	D	immédiats, branches, contrôle, constantes

3.3 Formats d'opérandes

Format	Taille totale	Complément
A	1	aucun
B16	3	`lvl:4 | disp:12`
B24	4	`lvl:4 | disp:20`
B40	6 si attribué	`lvl:4 | 0000 | disp:32`, réservé en V8
C24	4	`lvl:4 | ofs:4 | disp:16`
C32	5	`lvl:4 | ofs:8 | disp:20`
D8 / D16 / D24 / D32 / D64	2 / 3 / 4 / 5 / 9	immédiat de la largeur indiquée
D8_8	3	deux immédiats de 8 bits, `lsb` puis `w`
BR8 / BR16 / BR24 / BR32	2 / 3 / 4 / 5	déplacement relatif signé

Le champ `lvl` vaut 0 à 14 pour DISPLAY[LVL] et 15 pour une adresse prise sur la pile. `disp` est signé, sauf pour `LINK` où il représente une taille non signée. `ofs` est non signé.

3.4 Ordre des octets

Les compléments d'instruction sont rangés **poids fort en tête**. Cette règle ne doit pas être confondue avec le boutisme des données : les données mémoire restent petit-boutistes. Pour B et C, le premier octet complémentaire porte toujours `lvl` dans ses bits 7..4.

Instruction	Octets	Format
`LQ 3,-2049`	`5B 3F F7 FF`	B24
`LIB 1,8,16`	`98 11 00 00 08`	C32
`LI 128`	`C1 00 80`	D16
`UBFXI 12,20`	`C4 0C 14`	D8_8
`BT +32768`	`E6 00 80 00`	BR24

3.5 Immédiats et branches

Les immédiats ordinaires de `LI` sont signés et étendus à 64 bits. L'assembleur choisit la forme la plus étroite. La forme spéciale `LI imm4` est au contraire non signée et code directement 0 à 15 dans le bas de l'opcode.

Les déplacements des branches et de `CALL` sont relatifs à l'instruction suivante. Cette convention vaut pour BR8 à BR32 et pour le D24 de `CALL`.

Chapitre 4

Adressage, frames et accès mémoire

4.1 Calcul de l'adresse effective

Famille B - accès direct

`lvl = 0..14:` $EA = DISPLAY[lvl] + disp$
`lvl = 15:` $EA = pop + disp$

Famille C - accès indirect

`lvl = 0..14:` $EA = M64[DISPLAY[lvl] + disp] + ofs$
`lvl = 15:` $EA = M64[pop + disp] + ofs$

Avec `FMT=00`, il n'y a aucun complément : cela signifie `lvl=15`, `disp=0` et, pour C, `ofs=0`. Les formes les plus courtes servent donc naturellement aux accès dont l'adresse est déjà sur la pile.

4.2 Notation @ et @@

La spécification utilise `@` pour une adresse de donnée et `@@` pour l'adresse d'une cellule contenant elle-même une adresse.

Famille	Forme sur pile	Effet
B	`LVA`	`( @ - @+disp )`
B	`Lx`	`( @ - v )`
B	`Sx`	`( @ v - )`
C	`LIVA`	`( @@ - @ )`
C	`LIx`	`( @@ - v )`
C	`SIx`	`( @@ v - )`

Un rangement dépile d'abord la donnée, puis l'adresse placée juste dessous.

4.3 Chargements et rangements

Les tailles sont byte 8 bits, word 16 bits, double 32 bits et quad 64 bits. Les chargements de MODE 01 étendent le signe ; ceux de MODE 11 étendent par zéros. Les rangements écrivent seulement les bits de poids faible de la taille demandée. En 64 bits, `LA=LQ`, `SA=SQ`, `LIA=LIQ` et `SIA=SIQ`.

4.4 CHK

Les formes `CHKx` et `CHKIx` contrôlent une valeur déjà au sommet de la pile contre un couple de bornes lu en mémoire. Elles ne consomment pas la valeur : leur effet est `( v - v )`. Une valeur hors bornes lève la faute 131.

La famille de chargement choisit si les bornes mémoire sont signées ou non signées avant la comparaison signée sur 64 bits.

4.5 Replis d'assemblage

L'ISA V8 n'ajoute pas de format étendu. Lorsque `ofs` ne tient pas dans 0..255 ou que `disp` dépasse B24/C32, l'assembleur doit construire l'adresse avec plusieurs instructions et utiliser ensuite une forme `lvl=15`. Ces séquences sont des détails du code produit : elles n'ajoutent aucun état architectural.

Le format B40 est réservé pour une extension éventuelle de `LINK`, `EXC_MACH` et `LVA`. Tant qu'il n'est pas attribué, ses codes restent des opcodes réservés et lèvent la faute 137.

Chapitre 5

Instructions entières, logiques et de bits

5.1 Règle de débordement

Les entiers de calcul sont signés sur 64 bits. `ADD`, `SUB`, `MUL`, `NEG`, `ABS`, `INC` et `DEC` ne s'enveloppent jamais en silence. Si le résultat mathématique sort de $[-2^{63}, 2^{63})$, la faute 129 est livrée et les opérandes restent en place.

Les opérations logiques travaillent sur des motifs de bits et ne débordent pas. Les comptes de décalage doivent être compris entre 0 et 63.

5.1.1 Logique et décalages

Instruction	Effet de pile	Opération / remarques
`ET`	`( a b - a and b )`	ET bit à bit.
`OU`	`( a b - a or b )`	OU bit à bit.
`OUX`	`( a b - a xor b )`	OU exclusif bit à bit.
`NON`	`( a - not a )`	Complément bit à bit.
`SHL`	`( a n - a«n )`	Décalage logique gauche ; faute 137 si n >= 64.
`SHR`	`( a n - a»n )`	Décalage logique droite ; faute 137 si n >= 64.
`SAR`	`( a n - a»n )`	Décalage arithmétique droite ; faute 137 si n >= 64.
`CLAMP0`	`( a - max(0,a) )`	Remplace une valeur négative par zéro.

5.1.2 Comparaisons entières

Instruction	Effet de pile	Opération / remarques
`CGT`	`( a b - a>b )`	Résultat booléen 0/1.
`CLT`	`( a b - a<b )`	Résultat booléen 0/1.
`CNE`	`( a b - a/=b )`	Résultat booléen 0/1.
`CEQ`	`( a b - a=b )`	Résultat booléen 0/1.
`CGE`	`( a b - a>=b )`	Résultat booléen 0/1.
`CLE`	`( a b - a<=b )`	Résultat booléen 0/1.
`NEG`	`( a - -a )`	Faute 129 pour -2^63.
`ABS`	`( a - |a| )`	Faute 129 pour -2^63.

5.1.3 Arithmétique entière

Instruction	Effet de pile	Opération / remarques
ADD	(a b - a+b)	Addition signée contrôlée ; faute 129 en débordement.
INC	(a - a+1)	Incrément signé contrôlé.
SUB	(a b - a-b)	Soustraction signée contrôlée.
DEC	(a - a-1)	Décrément signé contrôlé.
MUL	(a b - a*b)	Produit signé ; faute 129 si le résultat exact ne tient pas sur 64 bits.
DIV	(a b - a/b)	Division signée, troncature vers zéro ; faute 128 si b=0, 129 pour -2^63/-1.
REMI	(a b - a rem b)	Reste de signe du dividende ; faute 128 si b=0.
MODI	(a b - a mod b)	Modulo de signe du diviseur ; faute 128 si b=0.

5.1.4 Champs de bits

Instruction	Effet de pile	Opération / remarques
UBFX	(v lsb w - champ)	Extrait w bits et étend par zéros.
SBFX	(v lsb w - champ)	Extrait w bits et étend le signe.
BFI	(old ins lsb w - new)	Insère w bits de ins dans old.
UBFXI	(v - champ)	Forme immédiate D8_8 : lsb puis w.
SBFXI	(v - champ)	Forme immédiate signée.
BFII	(old ins - new)	Forme immédiate d'insertion.

5.1.5 Virgule fixe

Instruction	Effet de pile	Opération / remarques
CVTIX	(i denom numer - x)	Calcule i*denom/numer sur produit exact 128 bits, troncature vers zéro ; fautes 128/129.
CVTXI	(x numer denom - i)	Calcule x*numer/denom sur produit exact, arrondi au plus proche, mi-chemin à l'écart de zéro ; fautes 128/129.

Chapitre 6

Instructions flottantes

6.1 Modèle IEEE 754

Les flottants sont des IEEE 754 binary64 transportés dans des cellules 64 bits de la pile data. `FADD`, `FSUB`, `FMUL`, `FDIV`, les multiplications de `FEXP` et `CVTIF` utilisent l'arrondi au plus proche, égalité vers le pair. Les sous-normaux sont traités normalement.

Les opérations arithmétiques flottantes ne lèvent pas de faute architecturale de virgule flottante : par exemple, une division par zéro peut rendre une infinité IEEE. Les conversions vers entier sont, elles, contrôlées.

Tout NaN produit par `FADD`, `FSUB`, `FMUL`, `FDIV` ou `FEXP` est remplacé par le NaN canonique `0x7FF8000000000000`. `FNEG` et `FABS` ne font que modifier le bit de signe et conservent la charge utile d'un NaN existant.

6.1.1 Arithmétique et conversions

Instruction	Effet de pile	Opération / remarques
`FADD`	`( a b - a+b )`	Addition binary64.
`FSUB`	`( a b - a-b )`	Soustraction binary64.
`FMUL`	`( a b - a*b )`	Multiplication binary64.
`FDIV`	`( a b - a/b )`	Division binary64.
`FEXP`	`( x n - x**n )`	Exponentiation par multiplications successives de gauche à droite ; durée proportionnelle à |n|.
`CVTIF`	`( i - f )`	Conversion entier signé 64 bits vers binary64.
`CVTFI`	`( f - i )`	Conversion vers entier par troncature ; faute 130 pour NaN ou valeur hors plage.
`CVTFIR`	`( f - i )`	Arrondi au plus proche, mi-chemin à l'écart de zéro ; faute 130 hors plage.

6.1.2 Comparaisons flottantes et signe

Instruction	Effet de pile	Opération / remarques
`FNEG`	`( f - -f )`	Inverse le bit 63.
`FABS`	`( f - |f| )`	Met le bit 63 à zéro.
`FCGT`	`( a b - a>b )`	0 pour une comparaison non ordonnée.
`FCLT`	`( a b - a<b )`	0 pour une comparaison non ordonnée.
`FCNE`	`( a b - a/=b )`	1 pour une comparaison non ordonnée, donc NaN /= x.
`FCEQ`	`( a b - a=b )`	0 pour une comparaison non ordonnée.
`FCGE`	`( a b - a>=b )`	0 pour une comparaison non ordonnée.
`FCLE`	`( a b - a<=b )`	0 pour une comparaison non ordonnée.

Chapitre 7

Pile, blocs et allocation

7.0.1 Manipulation de pile

Instruction	Effet de pile	Opération / remarques
`DROP`	`( a - )`	Supprime le sommet.
`DUP`	`( a - a a )`	Duplique le sommet.
`OVER`	`( a b - a b a )`	Recopie la cellule située juste sous le sommet.
`LI`	`( - imm )`	Empile une constante signée D8/D16/D32/D64 ; l'assembleur choisit la forme minimale.
`LI(imm4)`	`( - 0..15 )`	Empile directement le bas nibble non signé de l'opcode.

7.0.2 Blocs mémoire

Instruction	Effet de pile	Opération / remarques
`BLKMOV`	`( @dst len @src - )`	Copie len octets, sans recouvrement.
`BLKCMP`	`( @a len @b - eq )`	Rend 1 si les blocs sont égaux ; len=0 rend 1.
`BLKAND`	`( @dst len @src - )`	ET booléen octet par octet dans dst.
`BLKOU`	`( @dst len @src - )`	OU octet par octet dans dst.
`BLKOUX`	`( @dst len @src - )`	OU exclusif octet par octet dans dst.
`BLKNOT`	`( @dst len - )`	Chaque octet de dst est xoré avec 1.

7.0.3 Allocation

Instruction	Effet de pile	Opération / remarques
`CO_VAR`	`( n - @bloc )`	Arrondit n au multiple de 8 supérieur, rend l'ancien CSP puis avance CSP ; faute 135 sans effet si la limite serait dépassée.
`HEAP_ALLOC`	`( n - @bloc )`	Arrondit n au multiple de 8 supérieur, décrémente HP et rend le nouveau HP ; faute 136 sans effet sous LIM_HP.

7.1 Cellules de calcul et mémoire

La V8 formalise une propriété importante pour permettre une implantation efficace de la pile : une cellule de calcul peut être conservée en registre et son écriture mémoire peut être différée. Le programme HX produit par TLALOC ne doit pas écrire ces temporaires par un accès calculé. Une lecture calculée d'une cellule de calcul n'est définie que si un `LVA` connu l'a exposée depuis son dernier push. Le contenu au-dessus de DSP est indéfini.

Ces règles ne changent pas le modèle visible d'un programme Ada correct ; elles précisent les comportements qui ne doivent pas être exploités par du code HX fabriqué autrement.

Chapitre 8

Frames, appels et contrôle de flot

8.1 LINK et UNLINK

`LINK lvl,alloc` construit le frame. Pour un niveau supérieur à zéro, l'ancien DISPLAY[LVL] est empilé et le nouveau display pointe sur cette cellule ; l'espace local est ensuite réservé sur la pile data. Enfin, la valeur de CFP est chaînée sur la co-pile.

`UNLINK lvl` abandonne les locales et restaure le display et CFP, mais ne rend pas la co-pile. `UNLINKR lvl` rend en plus toute la co-pile du frame. Les niveaux autorisés pour les deux instructions de sortie sont 1 à 14 ; toute autre valeur lève la faute 137.

8.2 Appels et retours

Instruction	Effet	Opération
`CALL d`	`( - )`	empile l'adresse suivant l'instruction sur la pile des retours puis branche relativement
`CALLI`	`( @sp - )`	dépile la cible, empile le retour puis saute à la cible
`RTD n`	retour	retranche n octets à DSP puis dépile l'adresse de retour
`RTD`	retour	forme sans complément équivalente à RTD 0
`BRA d`	—	branche inconditionnelle
`BT d`	`( b - )`	branche si b est non nul
`BF d`	`( b - )`	branche si b vaut zéro

Les branches existent en BR8, BR16, BR24 et BR32. `CALL` utilise un déplacement D24 signé, soit une portée d'environ ±8 Mio d'octets.

8.3 EXC_MACH et EXC_RAISE

`EXC_MACH` enregistre dans un contexte mémoire DSP, RSP, CFP, CSP, le nombre de niveaux et les valeurs de DISPLAY[0..LVL]. Les champs PREV et DISPATCH du contexte sont remplis par du code LLIR ordinaire.

`EXC_RAISE` prend le contexte au sommet de la chaîne d'exception du niveau 0, le retire de la chaîne, restaure les pointeurs et le display, remet DR à zéro et saute au champ DISPATCH. Le tas n'est pas déroulé par les exceptions : HP n'est ni sauvé ni restauré.

Chapitre 9

Fautes, déroutements et interruptions

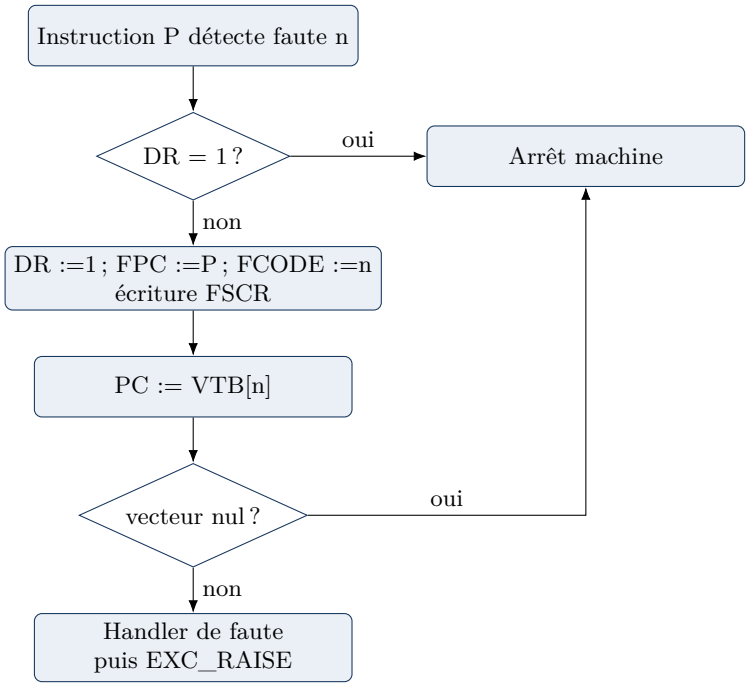
9.1 Trois sources de déroutement

TAHX_1 distingue trois catégories : les fautes synchrones détectées par la machine, les services synchrones demandés par `TRAP`, et les interruptions asynchrones provenant de la plateforme. Toutes utilisent un code 0..255 et une table de vecteurs VTB de 256 mots.

Codes	Usage
0..14	services TRAP
15	réservé
16..18	services machine CTX_SAVE, CTX_RESTORE, SET_IMASK
19..31	réservés services
32..63	interruptions externes
64..127	réservés
128..159	fautes
160..255	réservés

9.2 Faute précise

Une instruction fautive ne modifie aucun état architectural propre à son exécution. La livraison seule écrit DR, FPC, FCODE, la zone FSCR et PC. Le handler retrouve donc les opérandes en place et l'état d'avant l'instruction.



9.3 Table des fautes

Code	Exception Ada 83	Condition principale
128	NUMERIC_ERROR	division ou conversion fixe avec diviseur nul
129	NUMERIC_ERROR	débordement entier signé / résultat fixe non représentable
130	NUMERIC_ERROR	CVTFI/CVTFIR : NaN ou hors $[-2^{63}, 2^{63})$
131	CONSTRAINT_ERROR	CHK hors bornes
132	CONSTRAINT_ERROR	accès mémoire invalide, lecture d'instruction comprise
133	STORAGE_ERROR	dépassement de la pile data
134	STORAGE_ERROR	dépassement de la pile des retours
135	STORAGE_ERROR	dépassement de la co-pile
136	STORAGE_ERROR	épuisement du tas
137	PROGRAM_ERROR	opération indéfinie : opcode réservé, champs invalides, TRAP interdit, etc.
138..159	—	réservés

9.4 Priorité des fautes

Si plusieurs conditions pourraient être vraies pour la même instruction, une seule faute est livrée. L'ordre V8 est :

1. lecture invalide d'un octet d'instruction (132) ;
2. illégalité déductible des octets d'instruction (137) ;
3. limites DSP puis RSP (133, 134) ;
4. conditions sur les opérandes, CSP et HP (128, 129, 130, 137, 135, 136) ;
5. accès aux données (132) ;
6. conditions dépendant de valeurs lues, dont CHK (131) et certains services.

9.5 Réserves de piles

Quand DR=1, les handlers disposent de réserves architecturales qui retardent la faute de débordement : **1 Kio** pour la pile data, **32 adresses / 256 octets** pour la pile des retours et **1 Kio** pour la co-pile. Le tas n'a pas de réserve.

9.6 Interruptions

Les interruptions 32..63 sont présentées comme 32 requêtes pendantes. Si DR=0, le processeur choisit le plus petit code pendant non masqué, empile le PC suivant sur la pile des retours, expose le code dans FSCR, saute au vecteur et acquitte la requête. Le handler se termine par `RTX`, qui dépile le PC et remet DR à zéro.

Aucune interruption n'est livrée pendant une instruction ni tant que DR=1.

Chapitre 10

TRAP, services et contextes de tâche

10.1 TRAP

`TRAP n` adresse les services 0..14. Si le vecteur VTB[N] est non nul, l'instruction se comporte comme un appel vectorisé : l'adresse de retour est empilée et le service se termine par `RTX`. Si le vecteur est nul, le service peut être exécuté directement par l'hôte ou par la machine comme une instruction atomique. Un service absent ou un code non attribué lève la faute 137.

10.2 Services numérotés

Code	Service	Remarque
0	EXIT	`( code - )` ; la macro SYS_EXIT devient `LI code ; TRAP 0`
1	CLOCK_GETTIME	service de temps
2	PUT_CHAR	sortie caractère
3	PUT_STR	sortie chaîne
4	GET_CHAR	entrée caractère
5	GET_STR	entrée chaîne
6	FILE_CREATE	service fichier
7	FILE_OPEN	service fichier
8	FILE_SET_POS	service fichier
9	FILE_GET_POS	service fichier
10	FILE_GET_SIZE	service fichier
11	FILE_WRITE	service fichier
12	FILE_READ	service fichier
13	FILE_CLOSE	service fichier
14	FILE_DELETE	service fichier
16	CTX_SAVE	service machine
17	CTX_RESTORE	service machine
18	SET_IMASK	service machine

Périmètre de la V8

La V8 fixe la numérotation ci-dessus et donne explicitement les effets de pile des services machine, mais elle ne redécrit pas dans ce document l'ABI détaillée des services d'E/S 1..14 : elle renvoie aux effets des macros `SYS_xxx` de la référence LLIR. Ce manuel n'invente donc pas leurs paramètres.

10.3 CTX_SAVE et CTX_RESTORE

`CTX_SAVE` reçoit l'adresse d'un bloc de 192 octets et y sauve le PC de reprise, DSP, RSP, CFP, CSP, DR, les trois limites de piles et les quinze entrées de DISPLAY ; il rend 0. `CTX_RESTORE` restaure ce bloc, empile 1 et reprend au PC sauvegardé, selon une convention analogue à `setjmp/longjmp`.

10.4 SET_IMASK

`SET_IMASK` a l'effet `( m - ancien )`. Il charge les 32 bits de poids faible de IMASK, rend l'ancien masque et prend effet à partir de l'instruction suivante.

10.5 Lien avec les entrées de tâche Ada 83

Une interruption 32..63 peut être reliée à une entrée de tâche. Le run-time installe le vecteur et démasque l'interruption ; le handler enregistre l'appel d'entrée dans les structures du tasking, puis commute éventuellement de contexte au moyen de CTX_SAVE/CTX_RESTORE. Le rendez-vous Ada lui-même reste une fonction du run-time et n'est pas câblé dans l'ISA.

Chapitre 11

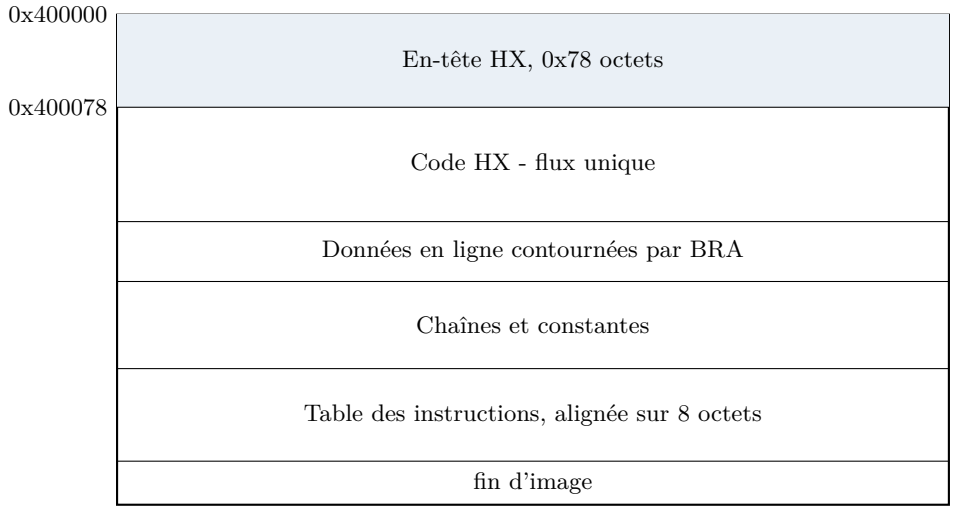
Image HX et démarrage

11.1 Format HX

Une image HX est chargée à l'adresse `0x400000`. Les mots de son en-tête sont petit-boutistes. Le code et les données commencent à `0x400078`.

Décalage	Champ	Signification
+0	`'TLALOCHX'`	signature, 8 octets
+8	version	1
+16	adresse de chargement	`0x400000`
+24	point d'entrée	première instruction, typiquement `0x400078`
+32	taille	code et données à partir de `0x400078`
+40	réservé TX	zéro en HX
+48	CEV	adresse de `STANDARD.ce_raise_`, recopiée en VTB[131] si non nulle
+56	table des instructions	adresse, ou zéro
+64	nombre d'instructions	nombre d'entrées de la table
+72	VTB	adresse d'une table de 256 vecteurs, ou zéro
+80	FSCR	zone de 16 octets, ou zéro
+88..+119	réservé	zéro jusqu'à `0x78`

11.2 Disposition conceptuelle



La table des instructions n'intervient jamais dans l'exécution matérielle. Elle aide l'émulateur à distinguer le code des données en ligne, à vérifier qu'un PC atteint un début d'instruction et à rattacher les séquences de repli HX à l'instruction LLIR correspondante.

11.3 État initial

Le chargeur initialise les piles, DISPLAY, la co-pile, le tas, les limites et l'état de déroutement. Au lancement : DR=0, FPC=0, FCODE=0 et toutes les interruptions sont masquées. Si VTB ou FSCR valent zéro dans l'en-tête, le chargeur fournit les zones correspondantes.

11.4 Bloc de démarrage de la plateforme

Le démarrage matériel n'est pas une instruction de l'ISA. La proposition de plateforme TAHX_1 consiste à lire, à une adresse `BOOT_BLOCK`, un bloc de 232 octets : le bloc de contexte de 192 octets suivi de HP, LIM_HP, VTB, FSCR et IMASK.

La plateforme présente aussi 32 bits `IRQ_PENDING` ; le processeur applique IMASK, choisit le plus petit code éligible et renvoie un acquittement avec le code livré.

Chapitre 12

Référence rapide des familles mémoire

12.1 Famille B : accès direct

Classe	Signé	Taille	Mnémotechniques
Load	oui	8/16/32/64	LB, LW, LD, LQ
Store	—	8/16/32/64	SB, SW, SD, SQ
Load	non	8/16/32	ULB, ULW, ULD
Check	oui	8/16/32/64	CHKB, CHKW, CHKD, CHKQ
Check	non	8/16/32	CHKUB, CHKUW, CHKUD
Spécial	—	—	LINK, EXC_MACH, LVA

Chaque charge/rangement ordinaire existe en forme FMT 00 (adresse sur pile, déplacement nul), B16 et B24. Les CHK utilisent B24.

12.2 Famille C : accès indirect

Classe	Signé	Taille	Mnémotechniques
Load	oui	8/16/32/64	LIB, LIW, LID, LIQ
Store	—	8/16/32/64	SIB, SIW, SID, SIQ
Load	non	8/16/32	ULIB, ULIW, ULID
Check	oui	8/16/32/64	CHKIB, CHKIW, CHKID, CHKIQ
Check	non	8/16/32	CHKUIB, CHKUIW, CHKUID
Spécial	—	adresse	LIVA

Les formes ordinaires sont FMT 00, C24 et C32 ; les CHKI utilisent C32.

12.3 Comparaison lexicographique

`LEXCMPx` et `ULEXCMPx` ont l'effet `( @g lg @d ld - r )` et rendent -1, 0 ou +1. Les longueurs sont en octets, ramenées à zéro par `CLAMP0` si nécessaire. Les variantes signées existent pour des composants 8, 16, 32 et 64 bits ; les variantes non signées pour 8, 16 et 32 bits, le signe étant indifférent en 64 bits pour cette forme.

Chapitre A

Carte complète de l'espace d'opcodes

Cette carte représente les 256 valeurs possibles du premier octet. RES signifie « réservé » du point de vue architectural. Les cases `D0` à `DF` codent la valeur immédiate 0 à 15 dans le nibble bas.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	ET	OU	OUX	NON	SHL	SHR	SAR	CLAMP0	NEG	CGT	CLT	CNE	CEQ	CGE	CLE	ABS
1	ADD	INC	SUB	DEC	MUL	DIV	REMI	MODI	UBFX	SBFX	BFI	RES	CVTIX	CVTXI	RES	RES
2	FADD	FSUB	FMUL	FDIV	FEXP	CVTIF	CVTFI	CVTFIR	FNEG	FCGT	FCLT	FCNE	FCEQ	FCGE	FCLE	FABS
3	DROP	DUP	OVER	CALLI	BLKMOV	BLKCMP	RES	RES	CO_VAR	HEAP_ALLOC	RES	RES	BLKAND	BLKOU	BLKOUX	BLKNOT
4	RES	RES	RES	RES	LINK	EXC_MACH	RES	LVA	LINK	EXC_MACH	RES	LVA	RES	RES	RES	RES
5	LB	LW	LD	LQ	LB	LW	LD	LQ	LB	LW	LD	LQ	CHKB	CHKW	CHKD	CHKQ
6	SB	SW	SD	SQ	SB	SW	SD	SQ	SB	SW	SD	SQ	RES	RES	RES	RES
7	ULB	ULW	ULD	RES	ULB	ULW	ULD	RES	ULB	ULW	ULD	RES	CHKUB	CHKUW	CHKUD	RES
8	RES	RES	RES	RES	RES	RES	RES	LIVA	RES	RES	RES	LIVA	RES	RES	RES	RES
9	LIB	LIW	LID	LIQ	LIB	LIW	LID	LIQ	LIB	LIW	LID	LIQ	CHKIB	CHKIW	CHKID	CHKIQ
A	SIB	SIW	SID	SIQ	SIB	SIW	SID	SIQ	SIB	SIW	SID	SIQ	RES	RES	RES	RES
B	ULIB	ULIW	ULID	RES	ULIB	ULIW	ULID	RES	ULIB	ULIW	ULID	RES	CHKUIB	CHKUIW	CHKUID	RES
C	LI	LI	LI	LI	UBFXI	SBFXI	BFII	RES	LEXCMPB	LEXCMPW	LEXCMPD	LEXCMPQ	ULEXCMPB	ULEXCMPW	ULEXCMPD	RES
D	LI 0	LI 1	LI 2	LI 3	LI 4	LI 5	LI 6	LI 7	LI 8	LI 9	LI 10	LI 11	LI 12	LI 13	LI 14	LI 15
E	BRA	BRA	BRA	BRA	BT	BT	BT	BT	BF	BF	BF	BF	RES	RES	RES	RES
F	TRAP	RES	CALL	RES	RES	RES	RTD	RTD	UNLINK	UNLINKR	RES	RES	RES	RES	EXC_RAISE	RTX

Codes réservés utilisés en interne

La V8 signale que l'implantation TAHX_1 peut réutiliser en forme décodée interne les valeurs C7, EC et ED pour des micro-opérations. Elles restent néanmoins **réservées dans le code programme** : un programme qui contient l'un de ces opcodes lève la faute 137.

Chapitre B

Table numérotée des instructions V8

La table suivante reprend les 180 formes numérotées de la table normative, en y ajoutant la valeur hexadécimale du premier octet. Pour `LI(imm4)`, le motif couvre la plage D0–DF.

No	OP	Fam.	Instruction	Format	Effet / remarque de la table V8
1	00	A	ET	—	(a b – a and b)
2	01	A	OU	—	(a b – a or b)
3	02	A	OUX	—	(a b – a xor b)
4	03	A	NON	—	(a – not a) complément bit à bit
5	04	A	SHL	—	(a n – a « n) faute 137 si n >= 64
6	05	A	SHR	—	(a n – a » n) logique, faute 137 si n >= 64
7	06	A	SAR	—	(a n – a » n) arithmétique, faute 137 si n >= 64
8	07	A	CLAMPO	—	(a – max(0,a))
9	08	A	NEG	—	(a – -a) case « jamais », faute 129
10	09	A	CGT	—	(a b – a>b) résultat 0/1
11	0A	A	CLT	—	(a b – a<b)
12	0B	A	CNE	—	(a b – a/=b)
13	0C	A	CEQ	—	(a b – a=b)
14	0D	A	CGE	—	(a b – a>=b)
15	0E	A	CLE	—	(a b – a<=b)
16	0F	A	ABS	—	(a – |a|) case « toujours », faute 129
17	10	A	ADD	—	(a b – a+b) faute 129
18	11	A	INC	—	(a – a+1) B := 0, retenue 1, faute 129
19	12	A	SUB	—	(a b – a-b) ~B, retenue 1, faute 129
20	13	A	DEC	—	(a – a-1) ~0, retenue 0, faute 129
21	14	A	MUL	—	(a b – a*b) produit signé, faute 129
22	15	A	DIV	—	(a b – a/b) troncature vers zéro, fautes 128, 129
23	16	A	REMI	—	(a b – a rem b) signe du dividende, faute 128
24	17	A	MODI	—	(a b – a mod b) signe du diviseur, faute 128
25	18	A	UBFX	—	(v lsb w – champ) extension par zéros, faute 137
26	19	A	SBFX	—	(v lsb w – champ) extension de signe, faute 137
27	1A	A	BFI	—	(old ins lsb w – new) faute 137
28	1C	A	CVTIX	—	(i denom numer – x) x = i*denom/numer, troncature vers zéro, fautes 128, 129
29	1D	A	CVTXI	—	(x numer denom – i) i = x*numer/denom arrondi, mi-chemin à l'écart de zéro, produit exact, fautes 128, 129
30	20	A	FADD	—	(a b – a+b)
31	21	A	FSUB	—	(a b – a-b)
32	22	A	FMUL	—	(a b – a*b)
33	23	A	FDIV	—	(a b – a/b)
34	24	A	FEXP	—	(x n – x**n) n entier, voir sémantique
35	25	A	CVTIF	—	(i – f)
36	26	A	CVTFI	—	(f – i) troncature vers zéro, faute 130
37	27	A	CVTFIR	—	(f – i) au plus proche, mi-chemin à l'écart de zéro (Ada RM 4.6), faute 130
38	28	A	FNEG	—	(f – -f) inversion du bit 63
39	29	A	FCGT	—	(a b – a>b)
40	2A	A	FCLT	—	(a b – a<b)
41	2B	A	FCNE	—	(a b – a/=b) 1 si non ordonné
42	2C	A	FCEQ	—	(a b – a=b)
43	2D	A	FCGE	—	(a b – a>=b)
44	2E	A	FCLE	—	(a b – a<=b)
45	2F	A	FABS	—	(f – |f|) mise à zéro du bit 63
46	30	A	DROP	—	(a –)
47	31	A	DUP	—	(a – a a)
48	32	A	OVER	—	(a b – a b a)
49	33	A	CALLI	—	(@sp –) adresse de retour sur pile des retours
50	34	A	BLKMOV	—	(@dst len @src –) sans recouvrement, len en octets
51	35	A	BLKCMP	—	(@a len @b – eq) 1 si égaux, len = 0 -> 1

No	OP	Fam.	Instruction	Format	Effet / remarque de la table V8
52	38	A	CO_VAR	—	(n – @bloc) CSP += n arrondi à 8, faute 135
53	39	A	HEAP_ALLOC	—	(n – @bloc) HP -= n arrondi à 8, faute 136
54	3C	A	BLKAND	—	(@dst len @src –) [dst] and= [src]
55	3D	A	BLKOU	—	(@dst len @src –) [dst] or= [src]
56	3E	A	BLKOUX	—	(@dst len @src –) [dst] xor= [src]
57	3F	A	BLKNOT	—	(@dst len –) chaque octet xor= 1 (NOT booléen)
58	44	B	LINK	B16	voir sémantique ; disp = taille locales, non signée (4 Kio)
59	45	B	EXC_MACH	B16	(–) disp = ctx
60	47	B	LVA	B16	(– @) ou (@ – @+disp) si lvl=1111
61	48	B	LINK	B24	taille non signée (1 Mio)
62	49	B	EXC_MACH	B24	—
63	4B	B	LVA	B24	—
64	50	B	LB	—	—
65	51	B	LW	—	—
66	52	B	LD	—	—
67	53	B	LQ	—	= LA
68	54	B	LB	B16	—
69	55	B	LW	B16	—
70	56	B	LD	B16	—
71	57	B	LQ	B16	—
72	58	B	LB	B24	—
73	59	B	LW	B24	—
74	5A	B	LD	B24	—
75	5B	B	LQ	B24	—
76	5C	B	CHKB	B24	—
77	5D	B	CHKW	B24	—
78	5E	B	CHKD	B24	—
79	5F	B	CHKQ	B24	—
80	60	B	SB	—	—
81	61	B	SW	—	—
82	62	B	SD	—	—
83	63	B	SQ	—	= SA
84	64	B	SB	B16	—
85	65	B	SW	B16	—
86	66	B	SD	B16	—
87	67	B	SQ	B16	—
88	68	B	SB	B24	—
89	69	B	SW	B24	—
90	6A	B	SD	B24	—
91	6B	B	SQ	B24	—
92	70	B	ULB	—	—
93	71	B	ULW	—	—
94	72	B	ULD	—	—
95	74	B	ULB	B16	—
96	75	B	ULW	B16	—
97	76	B	ULD	B16	—
98	78	B	ULB	B24	—
99	79	B	ULW	B24	—
100	7A	B	ULD	B24	—
101	7C	B	CHKUB	B24	—
102	7D	B	CHKUW	B24	—
103	7E	B	CHKUD	B24	—

No	OP	Fam.	Instruction	Format	Effet / remarque de la table V8
104	87	C	LIVA	C24	(– @) ou (@@ – @) si lvl=1111
105	8B	C	LIVA	C32	—
106	90	C	LIB	—	—
107	91	C	LIW	—	—
108	92	C	LID	—	—
109	93	C	LIQ	—	= LIA
110	94	C	LIB	C24	—
111	95	C	LIW	C24	—
112	96	C	LID	C24	—
113	97	C	LIQ	C24	—
114	98	C	LIB	C32	—
115	99	C	LIW	C32	—
116	9A	C	LID	C32	—
117	9B	C	LIQ	C32	—
118	9C	C	CHKIB	C32	—
119	9D	C	CHKIW	C32	—
120	9E	C	CHKID	C32	—
121	9F	C	CHKIQ	C32	—
122	A0	C	SIB	—	—
123	A1	C	SIW	—	—
124	A2	C	SID	—	—
125	A3	C	SIQ	—	= SIA
126	A4	C	SIB	C24	—
127	A5	C	SIW	C24	—
128	A6	C	SID	C24	—
129	A7	C	SIQ	C24	—
130	A8	C	SIB	C32	—
131	A9	C	SIW	C32	—
132	AA	C	SID	C32	—
133	AB	C	SIQ	C32	—
134	B0	C	ULIB	—	—
135	B1	C	ULIW	—	—
136	B2	C	ULID	—	—
137	B4	C	ULIB	C24	—
138	B5	C	ULIW	C24	—
139	B6	C	ULID	C24	—
140	B8	C	ULIB	C32	—
141	B9	C	ULIW	C32	—
142	BA	C	ULID	C32	—
143	BC	C	CHKUIB	C32	—
144	BD	C	CHKUIW	C32	—
145	BE	C	CHKUID	C32	—
146	C0	D	LI	D8	(– imm)
147	C1	D	LI	D16	—
148	C2	D	LI	D32	—
149	C3	D	LI	D64	LIF = LI du motif IEEE 754 lu signé (format le plus étroit)
150	C4	D	UBFXI	D8_8	(v – champ) lsb = imm, w = Imm (1..64)
151	C5	D	SBFXI	D8_8	(v – champ) extension de signe
152	C6	D	BFII	D8_8	(old ins – new)
153	C8	D	LEXCMPB	—	composants 8 bits signés
154	C9	D	LEXCMPW	—	16 bits signés
155	CA	D	LEXCMPD	—	32 bits signés

No	OP	Fam.	Instruction	Format	Effet / remarque de la table V8
156	CB	D	LEXCMPQ	—	64 bits
157	CC	D	ULEXCMPB	—	8 bits non signés (CHARACTER, BOOLEAN, énumérés)
158	CD	D	ULEXCMPW	—	16 bits non signés
159	CE	D	ULEXCMPD	—	32 bits non signés
160	D0-DF	D	LI(imm4)	—	(– imm4) imm4 non signé 0..15
161	E0	D	BRA	BR8	(–)
162	E1	D	BRA	BR16	—
163	E2	D	BRA	BR24	—
164	E3	D	BRA	BR32	—
165	E4	D	BT	BR8	(b –) saute si b /= 0
166	E5	D	BT	BR16	—
167	E6	D	BT	BR24	—
168	E7	D	BT	BR32	—
169	E8	D	BF	BR8	(b –) saute si b = 0
170	E9	D	BF	BR16	—
171	EA	D	BF	BR24	—
172	EB	D	BF	BR32	—
173	F0	D	TRAP	D8	service n, cf. « Fautes, déroutements et interruptions » §3
174	F2	D	CALL	D24	(–) déplacement relatif signé, ±8 Mio
175	F6	D	RTD	D24	DSP -= n (taille des paramètres), retour
176	F7	D	RTD	—	RTD 0
177	F8	D	UNLINK	D8	lvl 1..14 seulement, faute 137 sinon
178	F9	D	UNLINKR	D8	lvl 1..14 seulement, faute 137 sinon
179	FE	D	EXC_RAISE	D24	top = déplacement non signé dans le frame de niveau 0
180	FF	D	RTX	—	retour de service ou d'interruption, DR := 0

Chapitre C

Formats et longueurs - aide-mémoire

C.1 Formats

Format	Disposition après l'opcode
A	aucun complément
B16	`lvl[15:12] | disp[11:0]`
B24	`lvl[23:20] | disp[19:0]`
B40	`lvl[39:36] | 0000 | disp[31:0]` - réservé
C24	`lvl[23:20] | ofs[19:16] | disp[15:0]`
C32	`lvl[31:28] | ofs[27:20] | disp[19:0]`
D8/D16/D24/D32/D64	immédiat de 8/16/24/32/64 bits
D8_8	`imm[7:0] | Imm[7:0]`
BR8/BR16/BR24/BR32	déplacement relatif signé

C.2 Règles d'adresse

	lvl 0..14	**lvl=15**
B	$EA = DISPLAY[lvl] + disp$	$EA = pop + disp$
C	$EA = M64[DISPLAY[lvl] + disp] + ofs$	$EA = M64[pop + disp] + ofs$

C.3 Drapeaux et registres de déroutement

DR protège les handlers contre les interruptions imbriquées et transforme une seconde faute ou un service vectorisé en arrêt machine. FSCR est la seule exposition mémoire de FPC et FCODE : `M64[FSCR]=FPC`, `M64[FSCR+8]=FCODE`.

Chapitre D

Glossaire

LLIR	Représentation intermédiaire de bas niveau produite par la chaîne TLALOC.
HX	Format de code machine et format d'image exécutés par TAHX_1.
DSP	Data Stack Pointer, adresse de la cellule au sommet de la pile data.
RSP	Return Stack Pointer, adresse de la dernière adresse de retour empilée.
CSP	Co-stack Pointer, premier octet libre de la co-pile.
CFP	Co-stack Frame Pointer, tête du chaînage des frames de co-pile.
HP	Heap Pointer, frontière basse du tas alloué.
DISPLAY	Tableau des frame pointers de niveaux statiques 0..14.
VTB	Vector Table Base, base de la table de 256 vecteurs.
FSCR	Fault Status Communication Region, zone mémoire de 16 octets exposant FPC et FCODE.
DR	Diversion in progress, bit indiquant qu'un handler de faute ou d'interruption est en cours.
EA	Effective Address, adresse effective d'un accès mémoire.
FMT	Champ de format des familles B et C.
MODE	Champ sélectionnant spécial, chargement signé, rangement ou chargement non signé.

Référence normative

Spécification : *LLIR_hardware_support_V8.txt*, révision V8 du 29 septembre 2026, « Support matériel de la LLIR : machine à pile 64 bits ».
Référence exécutable indiquée par la V8 : `tx_run`, sous réserve de la liste de vérification de l'émulateur.
Licence du document source : GPL-3.0-or-later.

Index fonctionnel

Arithmétique entière	ADD, SUB, MUL, DIV, REMI, MODI, INC, DEC, NEG, ABS
Logique	ET, OU, OUX, NON, SHL, SHR, SAR
Champs de bits	UBFX, SBFX, BFI, UBFXI, SBFXI, BFII
Flottants	FADD, FSUB, FMUL, FDIV, FEXP, CVTIF, CVTFI, CVTFIR, FCxx
Pile	DROP, DUP, OVER, LI
Blocs	BLKMOV, BLKCMP, BLKAND, BLKOU, BLKOUX, BLKNOT
Allocation	CO_VAR, HEAP_ALLOC
Frames	LINK, UNLINK, UNLINKR, LVA, LIVA
Mémoire directe	LB/LW/LD/LQ, ULB/ULW/ULD, SB/SW/SD/SQ, CHKx
Mémoire indirecte	LIB/LIW/LID/LIQ, ULIB/ULIW/ULID, SIB/SIW/SID/SIQ, CHKIx
Contrôle	BRA, BT, BF, CALL, CALLI, RTD, TRAP, RTX
Exceptions	EXC_MACH, EXC_RAISE
Tasking / système	CTX_SAVE, CTX_RESTORE, SET_IMASK